BattleOps Week 5 Modules

PRIMARY OBJECTIVE: Break production-like systems intentionally, observe unpredictable failure behavior, design resilient recovery paths, validate DR strategies, diagnose cascading outages, and rehearse multi-zone failovers.

MONDAY: Failure Injection: PDB Collapse, Forced Shutdown, Pod Deathstorm

Critical Concepts

- PDB starvation under high load
- Cascading failures from forced pod terminations
- Slow drain → connection reset storms
- Failure prioritization logic inside kube-controller-manager
- How readiness + liveness collapse under stress

Key Outcomes

- Diagnose why PDBs **fail in real outages**
- Reproduce cascading workload collapse
- Understand shutdown races between kubelet, workload, and ingress
- Learn how applications behave when terminationGracePeriod is violated

WHAT TO PRACTICE

BattleLab 1: PDB Starvation Chaos

1. Create deployment with **PDB requiring 80% availability**
2. Force high CPU load on pods
3. Drain node
4. Observe: kubelet waits → retries → partial eviction → deadlock

You MUST explain: **Why draining does NOT proceed even if serviceload is high?**

BattleLab 2: Forced SIGKILL Chaos

1. Patch deployment with preStop hook (sleep 60)
2. Set terminationGracePeriod = 10
3. Delete pod
4. Observe: SIGTERM → incomplete draining → SIGKILL → connection burst failures

Analyse logs:

- Lost in-flight requests
- Readiness flaps
- Business-level error spikes

BattleLab 3: Pod Deathstorm Simulation

Write a loop killing 30 pods/second: while true; do kubectl delete pod -l app=test --force --grace-period=0; done

Observe:

- Scheduler backlog
- Container runtime exhaustion
- Node pressure conditions

Where to Study

- Kubelet lifecycle manager internals
- Graceful termination design docs
- PDB fairness & eviction manager behavior

InfraThrone Battle Mindset

“Your PDB does not save you, your shutdown logic does.”
“Eviction deadlocks are invisible until production burns.”

TUESDAY: Velero Corruption, SnapLoss, Cross-Cluster Restore

Critical Concepts

- Backup correctness vs backup success
- Snapshot consistency under heavy writes
- Recovery ordering (Secrets → PVC → Deployments → CRDs)
- Split-brain during multi-cluster restores
- Velero orphaned resource detection

Key Outcomes

- Simulate **data corruption despite successful backup**
- Execute cross-cluster restore under drift
- Validate backup completeness with integrity checks
- Debug snapshot chain failures

WHAT TO PRACTICE

BattleLab 1: "Backup Succeeded But Data Lost" Failure

1. Deploy MySQL or MongoDB with WAL/logging
2. Trigger constant writes
3. Take Velero backup WITHOUT Restic or snapshots
4. Blow away namespace
5. Restore
6. Observe: metadata restored, PVC restored empty → data loss scenario

Interview Question: **Why did backup show "Completed"?**

BattleLab 2: Snapshot Drift Chaos

1. Create a VolumeSnapshot
2. Keep writing to PVC during snapshot
3. Restore snapshot
4. Observe half-written, corrupted data

Explain:

- Crash consistency
- App-level flush requirements

BattleLab 3: Cross-Cluster Restore With Version Drift

Restore workloads into a cluster with:

- Different StorageClass
- Different CRD versions
- Missing Secrets

Analyze restore failure logs and dependency graph.

Where to Study

- Velero plugin architecture
- CSI Snapshot spec & failure modes
- StatefulSets restore order

InfraThrone Battle Mindset

“A backup is only real when the restore completes correctly.”
“Snapshots lie. Verifications do not.”

WEDNESDAY: Chaos Mesh & LitmusChaos: Network Partitions, Latency Cascades, API Server Storms

Critical Concepts

- Failure modes: network cut, CPU starvation, packet duplication
- Control-plane chaos (API server latency)
- Node-level chaos demons
- Latency cascading effects on microservices
- Blast radius design

Key Outcomes

- Create controlled-but-impactful chaos
- Map service dependencies during stress
- Detect cascading failures BEFORE total collapse
- Build automatic abort conditions for chaos experiments

WHAT TO PRACTICE

BattleLab 1: Network Partition Chaos

Split namespaces from each other:

- Block traffic between two services
- Observe retries, queue buildup, timeout storms
- Visualize dependency chain failure

BattleLab 2: API Server Latency Injection

Inject 500–2000ms latency to kube-apiserver:

- Watch how controllers lag
- HPA stops scaling
- PDB enforcement halts
- Pods reschedule late

This simulates: **High-load cluster under API bottleneck.**

BattleLab 3: Node CPU Meltdown

Use LitmusChaos to exhaust node CPU:

- Pods freeze
- Readiness drops
- Kubelet heartbeat delays
- Node eviction threshold triggered

Record: **Timeline of symptoms** → CPU → kubelet → pods → services.

Where to Study

- Litmus experiment docs
- Chaos Mesh network chaos CRDs
- Kube-apiserver internal queues

InfraThrone Battle Mindset

"Chaos is ordered only by observation."
"Your system resilience = your dependency understanding."

THURSDAY: Zonal Failures, NodeGroup Collapse, Storage Failover

Critical Concepts

- How workloads behave under zone outage
- Zonal storage vs regional storage realities
- NodeGroup draining under partial isolation
- Cross-zone network latency effects
- Zonal auto-scaler misbehavior

Key Outcomes

- Reproduce & understand **regional failover**
- Diagnose zonal workload imbalance
- Identify storage availability issues
- Validate topology constraints under failure

WHAT TO PRACTICE

BattleLab 1: Zone Blackout Simulation

Taint every node in zone A: kubectl taint node <zone-a-node> zone=out:NoSchedule

Observe:

- Pods recreate in zone B
- StatefulSet may NOT recover due to zonal PVCs
- Latency spikes across zone boundaries

BattleLab 2: NodeGroup Collapse

Simulate a scaling group failure:

- Remove auto-scaling from 1 zone
- Scale workloads
- Watch scheduler overcommit pods into single zone
- Combined load → cascading failures

BattleLab 3: Storage Failure Simulation

Detach PV volume during active I/O → Watch:

- Pod freeze
- App timeout
- Volume stuck in “Unknown”

Now failover to *regional* storage.

Where to Study

- GKE/AWS/Azure multi-zone docs
- How Load Balancers behave in zone failures
- Cloud provider storage replication

InfraThrone Battle Mindset

“Zonal HA is a lie if your storage is not regional.”
“Failover ≠ survival, recovery time matters.”

FRIDAY: Runbooks, Escalation Failure Chains & DR Drills Under Stress

Critical Concepts

- Incident priority classification
- Runbook execution under degraded conditions
- Escalation bottlenecks
- Failover with incomplete information
- DR automation gaps

Key Outcomes

- Build DR drills that reveal *real* gaps
- Execute failover with partial system visibility
- Analyze escalation delays
- Simulate human errors in DR workflow

WHAT TO PRACTICE

BattleLab 1: Broken Runbook Simulation

Use an intentionally outdated runbook.
Try to failover a service.

Document:

- Missing steps
- Wrong commands
- Assumptions that failed
- Data that was unavailable

This reveals operational rot.

BattleLab 2: Panic-Mode Multi-Service Failover

Simulate outage in:

- API layer
- Database layer
- Queue layer

Try to restore them in correct dependency order.

Evaluate:

- How long to identify root dependency?
- Which metrics mislead you?

BattleLab 3: Escalation Chain Stress Test

Simulate real company scenario:

- L1 sleeps
- L2 slow to respond
- L3 joins but has missing context

Time how long until the right engineer is engaged.

Where to Study

- Google SRE Incident Response
- PagerDuty failure stories
- NRC (normal, reversed, chaotic) escalation models

InfraThrone Battle Mindset

“A failover drill is successful only if it exposes pain.”
“Your recovery is only as strong as your worst runbook.”